



Anatomy of a Software Project

COP 4934

University of Central Florida

Rick Leinecker

Aspects of The Project

- Project Planning and Management
- Requirements Analysis
- Design and Prototyping
- Software Development
- Testing
- Deployment
- Documentation
- Evaluation and Maintenance

PLANNING

TEAM COLLABORATION

PLANNING

TEAM

COLLABORATION

RISKS

MILESTONES

RISKS

DELIVERABLES

SOFTWARE PROJECT

LIFECYCLE

CREATIVE

TIME

RISK

TIME

TIME

TIME

Anatomy of a Project

MONEY, TIME, TIME

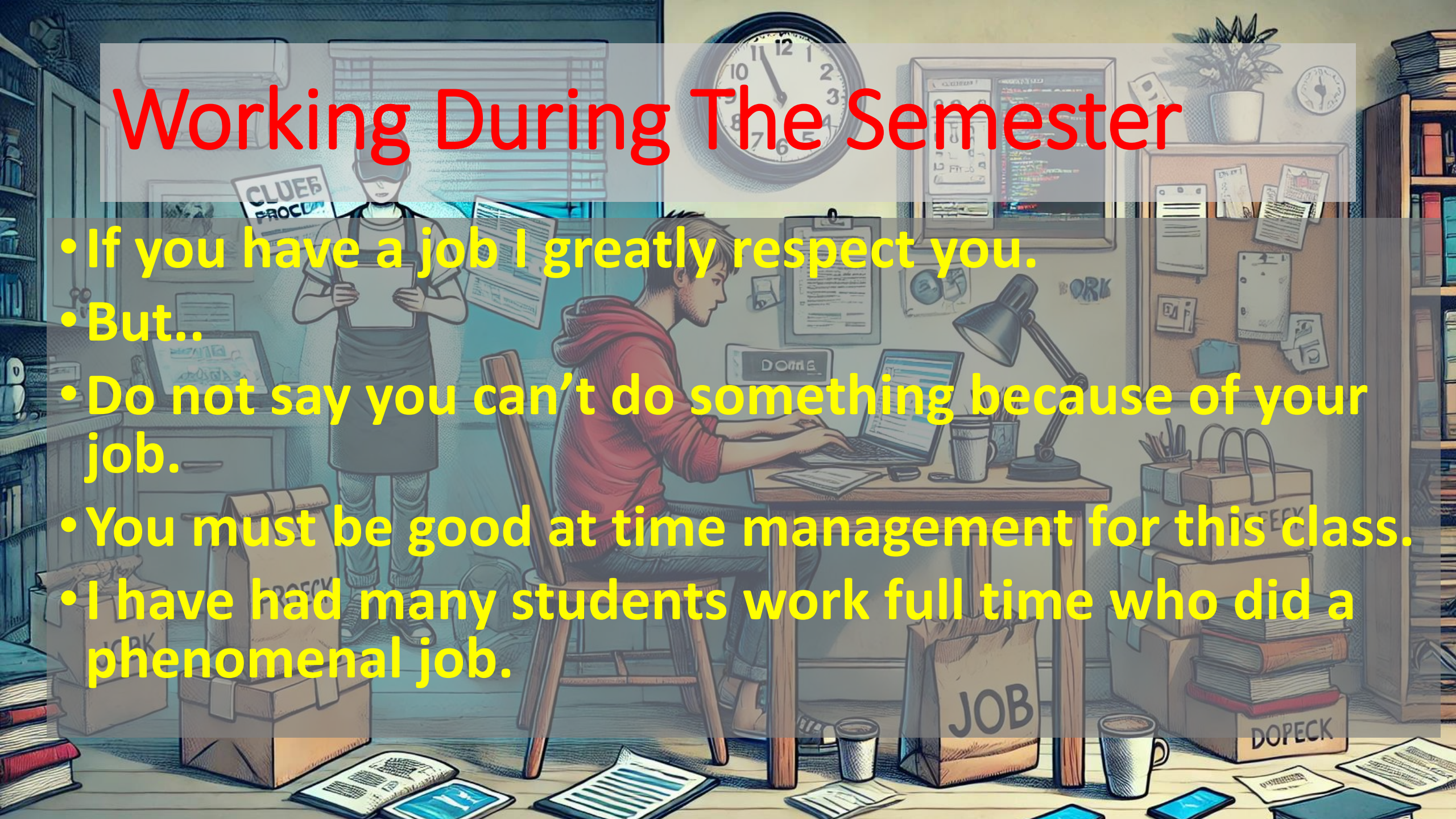


1. Project Planning and Management:

- **Scope Definition:** Clarifying what the project is supposed to achieve, including the goals, deliverables, tasks, costs, and deadlines.
- **Resource Allocation:** Determining the personnel, equipment, and materials necessary.
- **Scheduling:** Establishing a timeline for the completion of tasks.
- **Risk Management:** Identifying potential risks and devising contingency plans.

Working During The Semester

- If you have a job I greatly respect you.
- But..
- Do not say you can't do something because of your job.
- You must be good at time management for this class.
- I have had many students work full time who did a phenomenal job.



Project Managers



Educational Background

Relevant Experience

Project Management Skills

Technical Skills

Communication and Interpersonal Skills

Problem-Solving Skills

Financial and Resource Management

Adaptability and Flexibility

Software Management Don'ts

- Micromanaging the Team
- Ignoring Team Feedback
- Skipping Regular Check-ins
- Overlooking Risk Management
- Neglecting Stakeholder Engagement
- Underestimating Time and Resources
- Resisting Change
- Failing to Prioritize Tasks
- Lacking Documentation
- Ignoring Project Scope
- Compromising on Quality
- Neglecting Team Development
- Poor Conflict Resolution



2. Requirements Analysis:

- **Gathering Requirements:** Consulting with stakeholders to collect detailed information on the system's requirements.
- **Specification:** Creating a precise document that specifies every functional and non-functional requirement for the project.
- **Validation:** Ensuring the requirements meet the needs of the user and are technically feasible.

Minefields to Gathering Requirements

- Incomplete Requirements:
- Ambiguity:
- Over-specification:
- Scope Creep:
- Communication Barriers:
- Assuming User Needs:
- Lack of Stakeholder Involvement:
- Prioritization Issues:
- Technical Constraints:
- Cultural and Language Differences:
- Regulatory and Compliance Oversight:
- Resistance to Change:

3. Design and Prototyping

- **System Design:** Outlining the overall system architecture, including technology stack, system components, and their interactions.
- **Prototyping:** Developing a scaled-down model of the system to evaluate concepts or theories without the expense of system completion.
- **User Interface Design:** Designing the frontend of the application for optimal user experience.

Ideal System Design Components

- Purpose
- Diagrams such as a Use Case diagram.
- Executive summary.
- Technologies.
- Pipeline diagrams (This prevents you from getting yelled out in Senior Design 2).
- Data models and data acquisition, training, and accuracy
- Interface design including Figma diagrams
- Milestones
- Deployment

Prototyping

- Visualization
- Improve Understanding
- Early Feedback
- Reduce Miscommunications
- Test Usability
- Refine and Iterate
- Explore alternatives
- Technical Feasibility

User Interface Considerations

- User-Centered Design
- Usability
- Accessibility
- Responsiveness and Mobile First Design
- Performance Optimization
- Visual Design and Aesthetics
- Interaction Design
- Content Strategy
- Security Considerations
- Testing and Iteration
- Scalability

4. Software Development

- Coding: Actual writing of code in a suitable programming language. (Consider using a Linter)
- Integration: Combining and managing interactions between different pieces of software in the project.
- Development Methodologies: Agile, Waterfall, etc., that dictate how the development processes are undertaken. Agile (Scrum) is mandated.

5. Testing

- Unit Testing: Checking individual components for correct behavior.
- Integration Testing: Ensuring that combined components function together correctly.
- System Testing: Verifying that the system as a whole functions as intended.
- Acceptance Testing: Ensuring the system meets the requirements of the users.

DEPLOMENT
IN PROGRESS

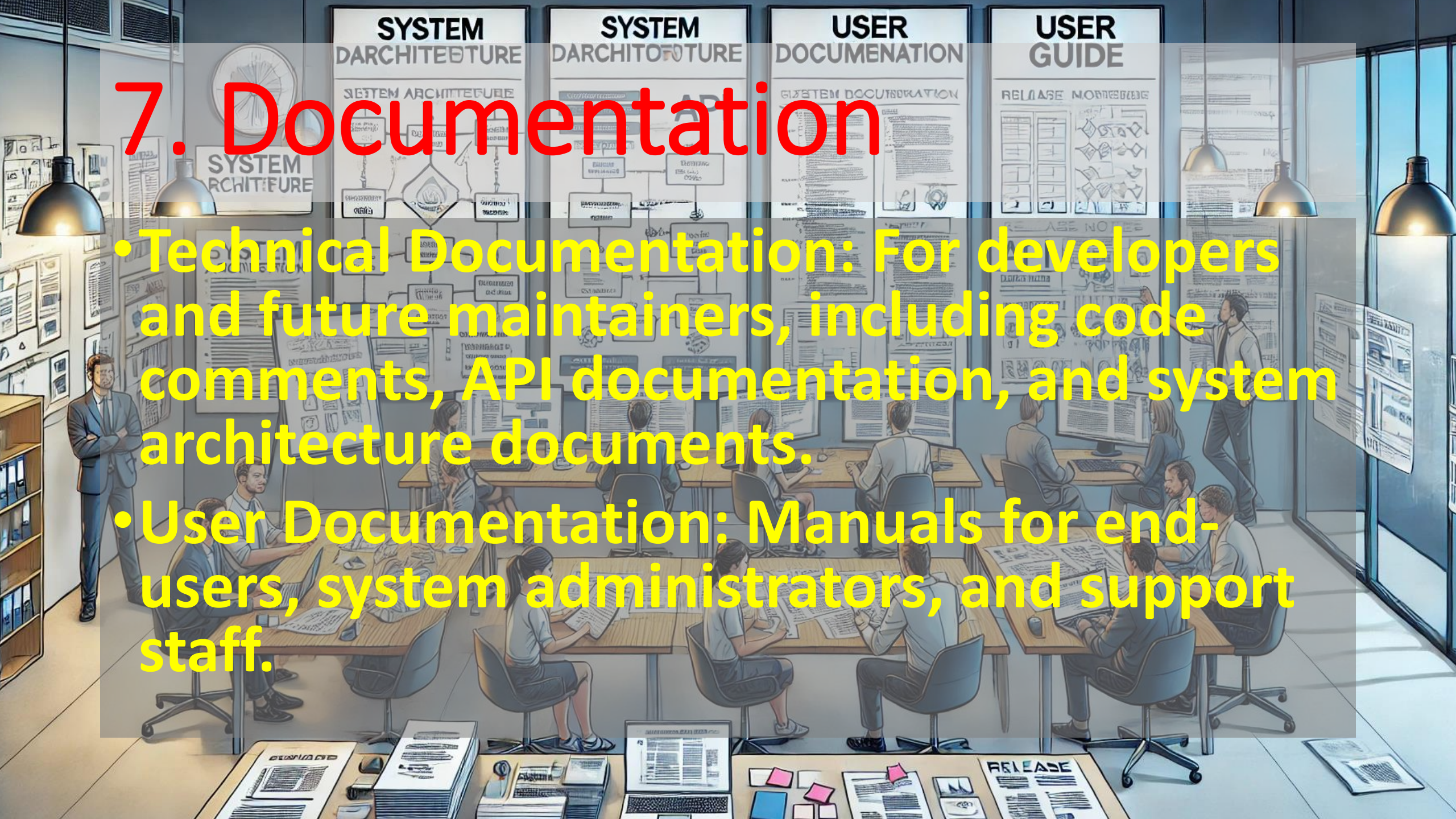
6. Deployment

DEPLOMENT
IN PROGRESS

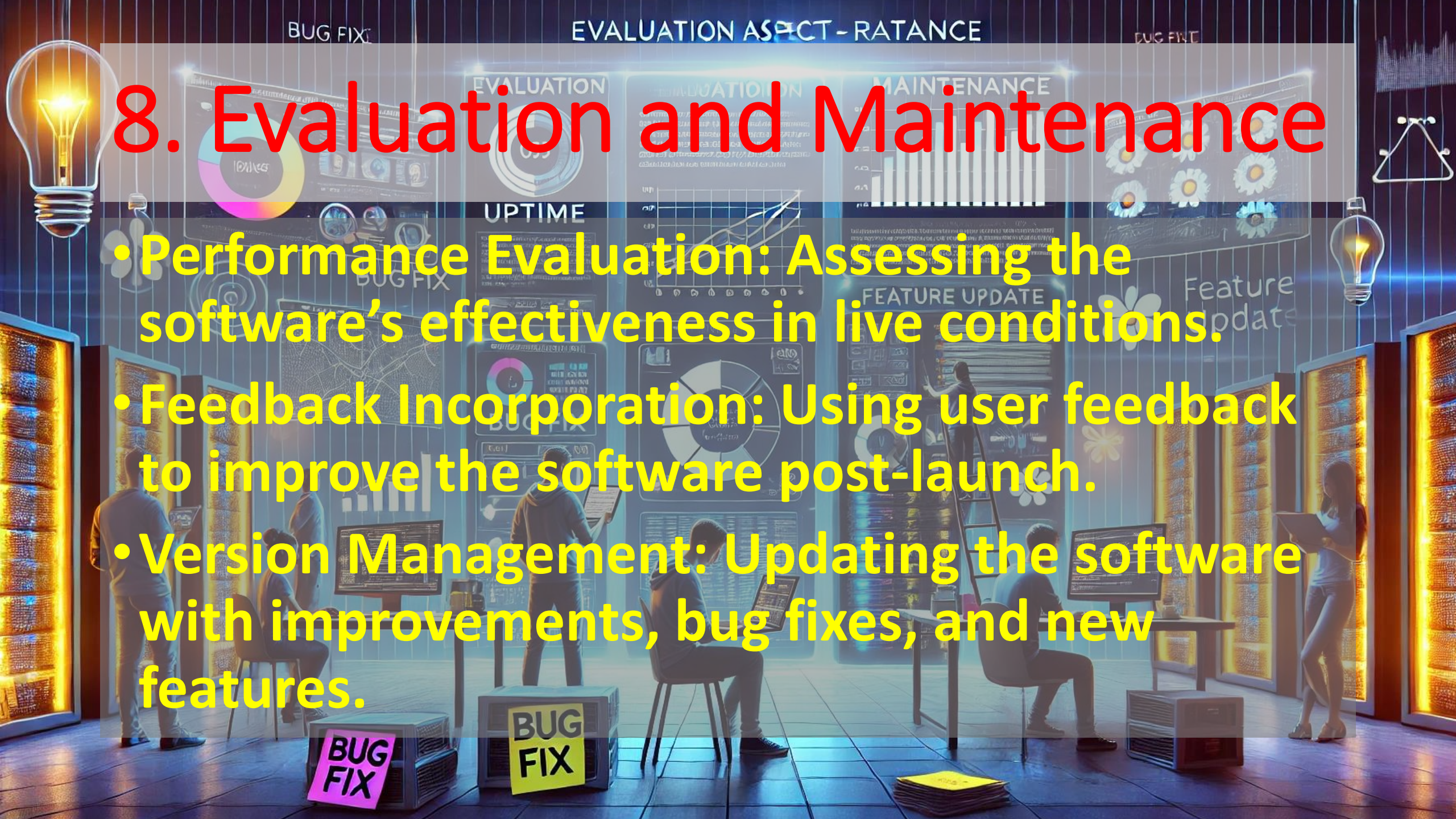
DEPLOMENT
IN PROGRESS
SUCCESSSS

- **Implementation:** The process of installing, configuring, and enabling a system in its production environment.
- **Migration:** Handling data and system transition from an old system to a new one without loss of information or service.
- **Support and Maintenance:** Providing ongoing assistance for the system and its users, and ongoing maintenance to deal with future problems that arise.

7. Documentation



- **Technical Documentation:** For developers and future maintainers, including code comments, API documentation, and system architecture documents.
- **User Documentation:** Manuals for end-users, system administrators, and support staff.



8. Evaluation and Maintenance

- **Performance Evaluation:** Assessing the software's effectiveness in live conditions.
- **Feedback Incorporation:** Using user feedback to improve the software post-launch.
- **Version Management:** Updating the software with improvements, bug fixes, and new features.

PROJECT COMPLETED
PROJECT COMPLETED
SUCCESSFULLY

Successful Project Completion

PROJECT
COMPLETED
SUCCESSFULLY

Thank You

Questions?

